

```

from flask import Flask
import pandas as pd
from google.cloud import bigquery
import smtplib
from email.mime.text import MIMEText
app = Flask(__name__)
def send_bigquery_email(request):
    try:
        PROJECT_ID = "ace-shine-465713-h2"
        DATASET = "analytics_494976509"
        TABLE = "bottlenecks_inference_table"
        TABLE1 = "events_intraday_20260112"
        SMTP_SERVER = "smtp.gmail.com"
        SMTP_PORT = 587
        EMAIL_USER = "sowmyar909@gmail.com"
        EMAIL_PASSWORD = "xxxx xxxx xxxx xxxx" # Gmail App Password
        EMAIL_TO = "sowmyar909@gmail.com"
        client = bigquery.Client()
        # -----
        # Query 1: Slow requests > 400ms
        # -----
        slow_query = f"""
        SELECT COUNT(*) AS slow_requests_count
        FROM `{PROJECT_ID}`.{DATASET}`.{TABLE1}`,
            UNNEST(event_params) AS event_params
        WHERE event_params.key LIKE '%Response_Time%'
            AND event_params.value.double_value > 0.400
        """
        slow_df = client.query(slow_query).to_dataframe()
        slow_requests = int(slow_df.iloc[0]["slow_requests_count"])
        # -----
        # Query 2: Response code counts
        # -----
        response_code_query = f"""
        SELECT
            event_params.value.int_value AS response_code,
            COUNT(*) AS request_count
        FROM `{PROJECT_ID}`.{DATASET}`.{TABLE1}`,
            UNNEST(event_params) AS event_params
        WHERE event_params.key LIKE '%Response_Code%'
        GROUP BY response_code
        ORDER BY response_code
        """
        rc_df = client.query(response_code_query).to_dataframe()
        passed_200 = int(rc_df.loc[rc_df["response_code"] == 200, "request_count"].sum())
        failed_requests = int(rc_df.loc[rc_df["response_code"] != 200, "request_count"].sum())
        # -----

```

```

# Table preview (sample rows)
# -----
table_query = f"""
SELECT * FROM `{PROJECT_ID}`.{DATASET}`.{TABLE}` LIMIT 10
"""

df = client.query(table_query).to_dataframe()
html_table = df.to_html(index=False) if not df.empty else "<p>No table data</p>"
# -----
# Single email body
# -----
html_body = f"""
<h2>JMeter API Performance Report</h2>
<h3>Summary</h3>
<ul>
    <li><b>Requests passed with 200 response code:</b> {passed_200}</li>
    <li><b>Requests failed (non-200 response codes):</b> {failed_requests}</li>
    <li><b>Requests slower than 400 ms:</b> {slow_requests}</li>
</ul>
<h3>Sample BigQuery Data</h3>
{html_table}
"""
# -----
# Send email
# -----
msg = MIMEText(html_body, "html")
msg["Subject"] = "JMeter Performance & BigQuery Report"
msg["From"] = EMAIL_USER
msg["To"] = EMAIL_TO
with smtplib.SMTP(SMTP_SERVER, SMTP_PORT) as server:
    server.starttls()
    server.login(EMAIL_USER, EMAIL_PASSWORD)
    server.send_message(msg)
return "Email sent successfully!"
except Exception:
    import traceback
    return f"Error:\n{traceback.format_exc()}", 500

```